Online gehen

Termin 5 · 27.10.2026

FÄLLIG
bis Termin 6 am 17.11. (drei Wochen)

ZEITBEDARF
ca. 3 Stunden, verteilt über drei Wochen

BAUSTEINE IM REPO
t5-pfade-und-fallen, t5-mini-server, t5-serverless-proxy

KONTAKT
dhbw@theaisoftwarecompany.com

Bis heute lief euer Projekt nur bei euch. Ab dieser Woche hat es eine Adresse, die ihr verschicken könnt. Danach habt ihr drei Wochen Zeit – nutzt sie, der Sprung zu JavaScript ist der größte im Semester.

Für die Pflicht reicht **statisches Hosting**: Eure Seite besteht aus HTML, CSS und JavaScript, die Daten holt der Browser direkt bei einer öffentlichen API. Einen eigenen Server braucht ihr erst, wenn ein Key geheim bleiben muss – das ist der Bonus in Teil B.

Teil A · Pflicht

A1 Deployen

Bringt euer Projekt auf **GitHub Pages**. Welcher Weg, hängt davon ab, ob ihr einen Build-Schritt habt.

Ohne Build (reines HTML, Tailwind per CDN):

1. Repo → Settings → Pages
2. Source: „Deploy from a branch", Branch `main`, Ordner `/ (root)`
3. eine Minute warten

Mit Vite (`npm run build` erzeugt `dist/`):

1. In `vite.config.js` den Repo-Namen eintragen: `base: "/<repo>/"`
2. Repo → Settings → Pages → Source: **GitHub Actions**
3. Den Workflow aus der Vite-Doku als `.github/workflows/deploy.yml` ins Repo legen (Kapitel „Deploying a Static Site" → GitHub Pages) und pushen
4. Im Tab *Actions* zuschauen, bis der Lauf grün ist

Den Quellordner direkt auszuliefern funktioniert bei Vite nicht – der Browser versteht `import "./style.css"` nicht. Online gehört nur `dist/`.

Wer lieber **Vercel**, **Netlify** oder **Cloudflare** nimmt: erlaubt. Repo verbinden, Vite wird erkannt, `base` und YAML entfallen. Dann steht diese URL in der README.

→ **Fertig, wenn** die Seite unter einer öffentlichen URL erreichbar ist und die URL in eurer README steht.

A2 Die eigene Seite im Network-Tab

Öffnet **eure deployte Seite** (nicht die lokale), DevTools → Network → neu laden.

Arbeitet die Liste ab:

- Gibt es rote Zeilen? Jede 404 hat einen Grund.
- Stimmen die Pfade? `/img/logo.png` zeigt bei GitHub Pages auf die Domain-Wurzel, nicht in euer Repo.
- Groß- und Kleinschreibung: `Logo.png` und `logo.png` sind lokal dasselbe, auf dem Server nicht.

→ **Fertig, wenn** beim Neuladen der deployten Seite **kein einziger** Request mit 4xx oder 5xx zurückkommt.

A3 Lokal und live vergleichen

Öffnet beide Fassungen nebeneinander. Notiert in der README drei Zeilen: Was war lokal anders als live, und woran lag es?

Falls nichts anders war: auch das ist ein Ergebnis – schreibt auf, welche drei Fallen ihr von vornherein vermieden habt.

→ **Fertig, wenn** die drei Zeilen stehen und konkret sind.

A4 README ausbauen

Eure README ist ab jetzt die Visitenkarte des Projekts:

- was die App kann, in drei bis fünf Punkten
- die Live-URL
- die Datenquelle (auch wenn sie erst in Termin 7 kommt)
- wie man das Projekt lokal startet
- ein Screenshot

→ **Fertig, wenn** jemand, der euer Projekt nicht kennt, nach 30 Sekunden Lesen weiß, worum es geht.

A5 Eure API prüfen – und keine Geheimnisse im Repo

Schaut euch die API an, die ihr in Termin 7 einbauen wollt, und ordnet sie ein:

- **Kein Key nötig** (z. B. PokéAPI, Open-Meteo, REST Countries) → passt zu GitHub Pages.
- **Öffentlicher Key**, den der Anbieter auf eure Domain einschränkt → darf ins Frontend.
- **Geheimer Key** (KI-, Bezahl-, Mail-APIs) → darf **nie** in den Browser, auch nicht als `VITE_`-Variable. Entweder eine andere API wählen oder Teil B machen.

Testet außerdem, ob die API CORS erlaubt: Öffnet eure **deployte** Seite, in der Console `await fetch("<api-url>")`. Kommt eine Antwort oder ein CORS-Fehler?

Durchsucht danach euer Repo nach API-Keys, Passwörtern und Tokens. Alles, was geheim ist, gehört in `.env`, und `.env` gehört in `.gitignore`.

→ **Fertig, wenn** in der README steht, welche API ihr nutzt, welche Key-Art sie hat und ob der CORS-Test geklappt hat – und ihr sagen könnt: In diesem öffentlichen Repo steht nichts, was nicht öffentlich sein darf.

Teil B · Bonus (+1 Punkt)

Zwei Wege, ihr wählt einen.

Weg 1 – Mini-Server lokal: Startet den Server aus `bausteine/t5-mini-server` und baut **eine eigene Route** dazu, die gefilterte Daten zurückgibt.

Weg 2 – Serverless Function online: Nehmt `bausteine/t5-serverless-proxy` als Vorlage und legt in eurem Projekt eine Function an (Vercel, Netlify oder Cloudflare), die eine fremde API aufruft und das Ergebnis weitergibt. Euer Frontend fragt nur noch `/api/...` – ein eventueller Key steht in den Einstellungen des Anbieters, nicht im Repo.

Für beide Wege in die README: in zwei Sätzen, wann ein eigener Server nötig ist und wann GitHub Pages reicht.

Teil C · Verständnisfragen

1. Was bedeuten die Statuscodes 200, 404 und 500 – und wer ist bei welchem „schuld"?
2. Eure Seite läuft lokal und live nicht. Nennt drei mögliche Ursachen und je einen Weg, sie zu prüfen.
3. Was ist der Unterschied zwischen einer Datei, einem lokalen Server und einem Hosting?
4. Was ist CORS, wen schützt es – und warum könnt ihr einen CORS-Fehler nicht im eigenen Frontend-Code beheben?
5. Ihr wollt speichern, welche Karten ein Nutzer als Favorit markiert hat. Nehmt ihr localStorage oder ein Cookie? Begründet mit einem Unterschied zwischen beiden.

So arbeitest du hier mit KI

Diese Woche ist die KI besonders schlecht in genau eurem Problem: Sie sieht euren Network-Tab nicht.

Deshalb bringt ihr die Beweise mit:

> *„Meine Seite läuft lokal, live nicht. Im Network-Tab bekomme ich für `/img/logo.png` einen 404. Die Seite liegt unter `benutzer.github.io/projekt/` . Erklär mir, warum der Pfad lokal geht und dort nicht – aber sag mir noch nicht die Lösung, ich will selbst drauf kommen."*

Merkt euch das Muster: **Was ihr erwartet habt, was passiert ist, was ihr schon geprüft habt.** So formuliert man Fehlerberichte auch im Job.